OmniDev Integration Engineer

Make multiple projects work together with evidence, clear ownership and reviewable changes. Updated October 9, 2026. This guide describes the implemented workflow and the remaining live verification requirements. It does not certify production interoperability.

What it now does

Select 2-8 imported projects in one batch. OmniDev records their source digests, separates their source namespaces, extracts supplied JSON OpenAPI operations, and flags differing dependency version strings. A difference is a review signal, not proof that a dependency is incompatible. The assigned Integration Engineering brain receives the project evidence and your objective. It can propose adapters, contract artifacts, documentation and verification tests. A joint proposal can be split into separate candidates for each original repository. Original sources are preserved until you choose an apply action.

Use it inside OmniDev

1. Import the projects you want to connect. Use the project selector to choose the first one.

2. Open Development. In Integration Engineering, select the other projects. Ctrl-click on Windows or Command-click on Mac selects several.

3. Write what should happen in the objective box: for example, send completed orders to billing and customer analytics.

4. Choose Map integration. Review the observed contracts, version differences and unanswered ownership/authentication questions.

5. Choose Generate integration changes. Your assigned brain proposes bounded changes using the available source.

6. Review the joint candidate, then choose Stage changes for original repositories. Each candidate can be reviewed and tested separately.

7. Apply only after acceptable evidence. A draft pull request is a separate action; staging and snapshot application do not deploy your apps. Large integrations require batches: the current combined source budget is 320 files / 2.5 MB, and each model has an additional context limit. This is not an unlimited full-source merge.

What Docker means

Docker runs a job in a separate container. The computer supplies the CPU, memory and storage.

The free software option

Docker Engine is an open-source container engine available for Linux. Existing computer hardware can run it; a rented cloud server is a separate cost. Docker Desktop for Windows/Mac has its own commercial eligibility rules. OmniDev does not require Vercel or a paid Docker subscription for the Linux Engine path.

What OmniDev adds

A resource-aware worker queue, up to ten concurrent slots on sufficient hardware, digest-bound source and command assignments, offline dependency installation, constrained disposable execution, output limits, cleanup checks and duplicate-claim protection. These are OmniDev features built above the container engine. The scheduler reserves 30% of host CPU and memory. Each application slot budgets two CPUs and 1 GB. A 32-CPU / 16-GB host can admit ten slots under this model. That is a configured concurrency limit, not a measured tenfold speed guarantee. Use one worker process per host.

Worker activation requirements

A dedicated Docker-capable Linux host, or an eligible Docker Desktop setup. Node.js and the OmniDev worker package. A pre-pulled, approved image pinned to its SHA-256 digest. Application jobs also need the exact offline dependency cache. Owner-private Site access, a worker authorization token and the Site service token configured outside source control. Complete source, package.json, package-lock.json and meaningful tests for each Node application being tested. A successful doctor check and real-container qualification before treating the worker as ready.

Why it is not running in this authoring workspace

Docker is absent here. The actual rootless permission probe failed with Operation not permitted while mapping a user namespace. Installing an executable would not supply the missing host permission or create a qualified isolated environment.

Verification and remaining work

What was tested

15 SQLite and scheduler checks covered a three-project evidence map, candidate splitting, repeat staging, unchanged originals, stale-source rejection, shared graph relationships, concurrency limits, host backpressure and duplicate execution claim rejection. Forty simulated queued jobs obeyed a three-slot budget. These are real control-flow checks with fixture application data, not live interoperability or container isolation proof.

What still needs live evidence

Run actual app builds and tests on an attached worker. Verify consumer/provider contracts, identity and tenant boundaries, duplicate events, timeouts, partial failure and recovery in a qualified multi-service test topology. The current Node worker tests each supplied application bundle independently; it does not provision databases or certify deployed connectivity between apps. JSON OpenAPI extraction is implemented. YAML, GraphQL, gRPC, AsyncAPI and runtime-trace extraction remain explicit extension work. Unknown contracts are not treated as absent functionality. The 48 specialist areas have bounded review/proposal workflows; external domain tools and production expertise remain subject to their own qualification.

Source of truth

Current code, tests and evidence are stored with the OmniDev source. Open qualification requirements are recorded under UNFINISHED-WORK, including integration-fleet-current.md and specialist-48-status.md.

Official references

Docker Engine installation: https://docs.docker.com/engine/install/ Rootless Docker requirements: https://docs.docker.com/engine/security/rootless/ Container resource limits: https://docs.docker.com/engine/containers/resource_constraints/ Docker Desktop eligibility: https://docs.docker.com/subscription-billing/desktop-license/ OpenAPI contract specification: https://spec.openapis.org/oas/v3.1.0.html